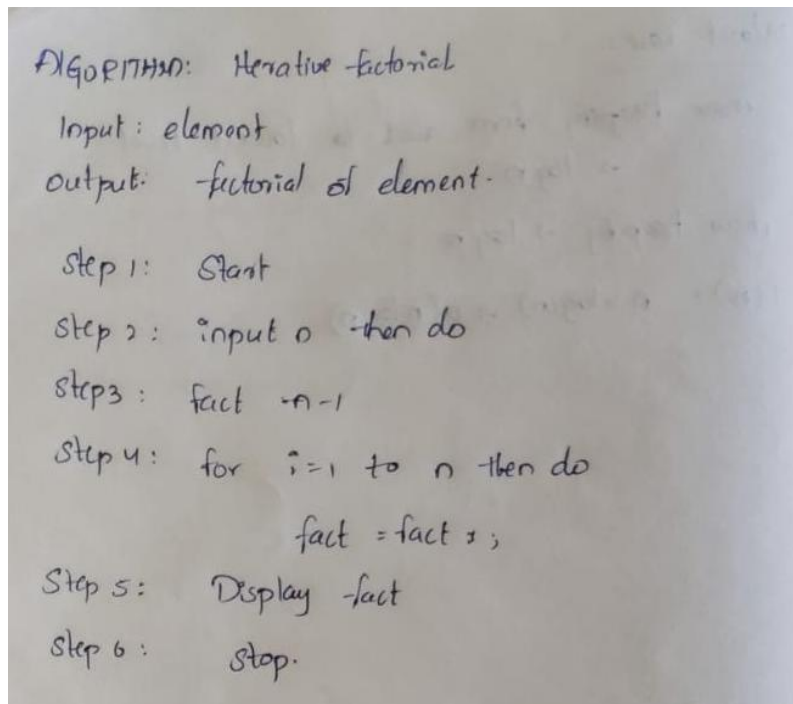




PRACTICAL: 04

Aim: Implementation and Time analysis of factorial program using iterative and recursive method.

Algorithm:



ALGORITHM: Iterative Factorial
Input: element
output: factorial of element
Step 1: Start
Step 2: input n then do
Step 3: fact = 1
Step 4: for i = 1 to n then do
 fact = fact * i;
Step 5: Display fact
Step 6: Stop.

Program / Code:

```
#include <iostream>
using namespace std;
class Factorial
{
public:
    int fact(int n)
    {
        int result = 1;
```



```
for (int i = 1; i <= n; i++)
{
    result = result * i;
}
return result;
}
};

int main()
{
    int n, result;
    Factorial obj;
    cout << "Enter the value to find its factorial: ";
    cin >> n;
    result = obj.fact(n);
    cout << "Factorial of " << n << " is " << result << endl;
    return 0;
}
```

Output:

```
PS C:\Users\pusap\Downloads\c++> C:\Users\pusap\Downloads\c++\factorial_iterative.exe
Enter the value to find its factorial: 4
Factorial of 4 is 24
```



Time Complexity:

Time Complexity:

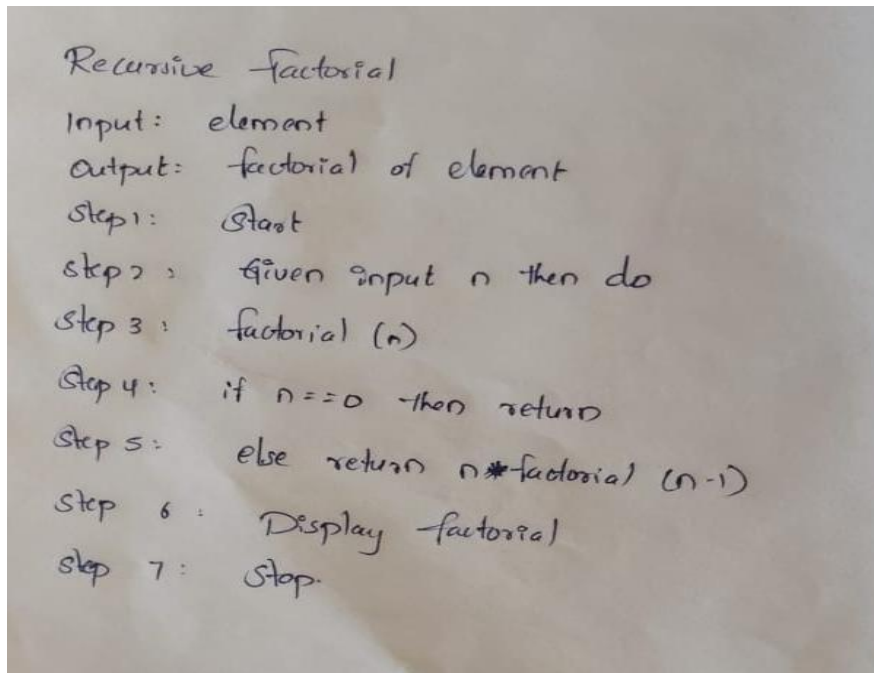
```
for (i = 1 ; i <= n ; i++) → . n
    f
    fact = fact * i ; → n
    g
```

Best case:	Average case	worst case
$T(n) = n+1+n$		
$= 2n+1$	$T(n) = 2n+2$	$T(n) = 2n+2$
$T(n) = O(n)$	$T(n) = O(n)$	$T(n) = O(n)$
if $n = 0$		
$T(n) = O(1)$		



RECURSIVE METHOD:

Algorithm:



Program:

```
#include<iostream>
using namespace std;

int factorial(int n) {
    if (n == 0 || n == 1)
        return 1;
    else
        return n * factorial(n - 1);
}

int main() {
```



```
int num;

cout << "Enter a number: ";
cin >> num;

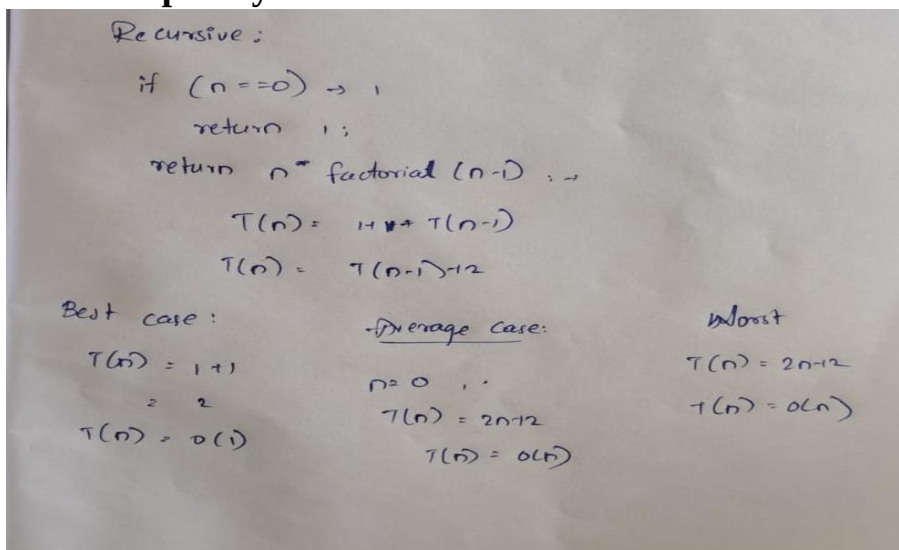
int result = factorial(num);
cout << "Factorial of " << num << " is " << result << endl;

return 0;
}
```

Output:

```
PS C:\Users\pusap\Downloads\c++> C:\Users\pusap\Downloads\c++\factorial.exe
Enter a number: 3
Factorial of 3 is 6
```

Time complexity:





Conclusion:

The factorial program was implemented using both iterative and recursive methods. The iterative method uses a loop, while the recursive method uses function calls. Both methods have $O(n)$ time complexity in the general case. The iterative method uses $O(1)$ space, whereas the recursive method uses $O(n)$ space due to the recursive call stack.